

Introduction to PyTorch - II

PyTorch is an open-source machine learning framework based on the Python programming language, used for building and training deep learning models. Widely adopted across academia and industry, PyTorch has become the framework of choice for cutting-edge research and commercial AI applications.

Build the Neural Network

Neural networks comprise of layers/modules that perform operations on data. The [torch.nn](#) namespace provides all the building blocks you need to build your own neural network. Every module in PyTorch subclasses the [nn.Module](#). A neural network is a module itself that consists of other modules (layers). This nested structure allows for building and managing complex architectures easily.

In the following sections, we'll build a neural network to classify images in the FashionMNIST dataset.

```
import os
import torch
from torch import nn
from torch.utils.data import DataLoader
from torchvision import datasets, transforms
```

Get Device for Training:

```
device = torch.accelerator.current_accelerator().type if torch.accelerator.is_available() else "cpu"
print(f"Using {device} device")
```

Out:

```
Using cpu device
```

Define the Class:

We define our neural network by subclassing **nn.Module**, and initialize the neural network layers in **__init__**. Every **nn.Module** subclass implements the operations on input data in the forward method.

```

class NeuralNetwork(nn.Module):
    def __init__(self):
        super().__init__()
        self.flatten = nn.Flatten()
        self.linear_relu_stack = nn.Sequential(
            nn.Linear(28*28, 512),
            nn.ReLU(),
            nn.Linear(512, 512),
            nn.ReLU(),
            nn.Linear(512, 10),
        )

    def forward(self, x):
        x = self.flatten(x)
        logits = self.linear_relu_stack(x)
        return logits

```

We create an instance of NeuralNetwork, and move it to the device, and print its structure.

```

model = NeuralNetwork().to(device)
print(model)

```

```

NeuralNetwork(
  (flatten): Flatten(start_dim=1, end_dim=-1)
  (linear_relu_stack): Sequential(
    (0): Linear(in_features=784, out_features=512, bias=True)
    (1): ReLU()
    (2): Linear(in_features=512, out_features=512, bias=True)
    (3): ReLU()
    (4): Linear(in_features=512, out_features=10, bias=True)
  )
)

```

To use the model, we pass it the input data. This executes the model's forward.

Calling the model on the input returns a 2-dimensional tensor with dim=0 corresponding to each output of 10 raw predicted values for each class, and dim=1 corresponding to the individual values of each output. We get the prediction probabilities by passing it through an instance of the nn.Softmax module.

```

X = torch.rand(1, 28, 28, device=device)
logits = model(X)
pred_probab = nn.Softmax(dim=1)(logits)
y_pred = pred_probab.argmax(1)
print(f"Predicted class: {y_pred}")

```

```

Predicted class: tensor([1])

```

Model Layers:

Let's break down the layers in the FashionMNIST model. To illustrate it, we will take a sample minibatch of 3 images of size 28x28 and see what happens to it as we pass it through the network.

```

input_image = torch.rand(3,28,28)
print(input_image.size())

```

Out:

```

torch.Size([3, 28, 28])

```

nn.Flatten:

We initialize the `nn.Flatten` layer to convert each 2D 28x28 image into a contiguous array of 784 pixel values.

```
flatten = nn.Flatten()
flat_image = flatten(input_image)
print(flat_image.size())
```

Out:

```
torch.Size([3, 784])
```

nn.Linear:

The `linear layer` is a module that applies a linear transformation on the input using its stored weights and biases.

```
layer1 = nn.Linear(in_features=28*28, out_features=20)
hidden1 = layer1(flat_image)
print(hidden1.size())
```

Out:

```
torch.Size([3, 20])
```

nn.ReLU:

Non-linear activations are what create the complex mappings between the model's inputs and outputs. They are applied after linear transformations to introduce *nonlinearity*, helping neural networks learn a wide variety of phenomena.

In this model, we use `nn.ReLU` between our linear layers, but there's other activations to introduce non-linearity in your model.

```
print(f"Before ReLU: {hidden1}\n\n")
hidden1 = nn.ReLU()(hidden1)
print(f"After ReLU: {hidden1}")
```

Out:

```
Before ReLU: tensor([[ 0.2325,  0.1103, -0.2654, -0.1363, -0.9310, -0.0548,  0.2203,  0.2287,
  0.3633,  0.3511, -0.0315,  0.3447, -0.4546, -0.1788,  0.7840, -0.6739,
  0.4063, -0.5974, -0.4943, -0.3409],
 [ 0.3870,  0.0315, -0.1706,  0.1517, -0.4887,  0.0921,  0.0470,  0.1635,
  0.3492,  0.4029,  0.3045,  0.2754, -0.0296,  0.0372,  0.3782, -0.5806,
  0.5079, -0.4094, -0.5246, -0.2348],
 [ 0.2037, -0.2825, -0.1566, -0.0679, -0.4086,  0.0572,  0.2086,  0.4646,
  0.0730,  0.1096, -0.1597,  0.5042, -0.4850, -0.0994,  0.7446, -0.8548,
  0.3735, -0.2384, -0.2991, -0.2548]], grad_fn=<AddmmBackward0>)
```

```
After ReLU: tensor([[0.2325, 0.1103, 0.0000, 0.0000, 0.0000, 0.0000, 0.2203, 0.2287, 0.3633,
 0.3511, 0.0000, 0.3447, 0.0000, 0.0000, 0.7840, 0.0000, 0.4063, 0.0000,
 0.0000, 0.0000],
 [0.3870, 0.0315, 0.0000, 0.1517, 0.0000, 0.0921, 0.0470, 0.1635, 0.3492,
 0.4029, 0.3045, 0.2754, 0.0000, 0.0372, 0.3782, 0.0000, 0.5079, 0.0000,
 0.0000, 0.0000],
 [0.2037, 0.0000, 0.0000, 0.0000, 0.0000, 0.0572, 0.2086, 0.4646, 0.0730,
 0.1096, 0.0000, 0.5042, 0.0000, 0.0000, 0.7446, 0.0000, 0.3735, 0.0000,
 0.0000, 0.0000]], grad_fn=<ReluBackward0>)
```

nn.Sequential:

`nn.Sequential` is an ordered container of modules. The data is passed through all the modules in the same order as defined. You can use sequential containers to put together a quick network like `seq_modules`.

```
seq_modules = nn.Sequential(
    flatten,
    layer1,
    nn.ReLU(),
    nn.Linear(20, 10)
)
input_image = torch.rand(3,28,28)
logits = seq_modules(input_image)
```

nn.Softmax:

The last linear layer of the neural network returns *logits* - raw values in $[-\infty, \infty]$ - which are passed to the `nn.Softmax` module. The logits are scaled to values $[0, 1]$ representing the model's predicted probabilities for each class. `dim` parameter indicates the dimension along which the values must sum to 1.

```
softmax = nn.Softmax(dim=1)
pred_probab = softmax(logits)
```

```
print(pred_prob)
✓ 0.0s
tensor([[0.1166, 0.0952, 0.0948, 0.1008, 0.0936, 0.0804, 0.0843, 0.1426, 0.1037,
 0.0880],
 [0.1258, 0.0989, 0.0935, 0.0853, 0.0905, 0.0885, 0.0890, 0.1296, 0.1169,
 0.0820],
 [0.1158, 0.1026, 0.0948, 0.1037, 0.0930, 0.0720, 0.0828, 0.1480, 0.0997,
 0.0876]], grad_fn=<SoftmaxBackward0>)
```

```
y_pred = pred_prob.argmax(1)
print(y_pred)
✓ 0.0s
```

```
tensor([7, 7, 7])
```

Model Parameters:

Many layers inside a neural network are *parameterized*, i.e. have associated weights and biases that are optimized during training. Subclassing `nn.Module` automatically tracks all fields defined inside your model object, and makes all parameters accessible using your model's `parameters()` or `named_parameters()` methods.

In this example, we iterate over each parameter, and print its size and a preview of its values.

```
print(f"Model structure: {model}\n\n")

for name, param in model.named_parameters():
    print(f"Layer: {name} | Size: {param.size()} | Values : {param[:2]} \n")
```

```
Model structure: NeuralNetwork(
  (flatten): Flatten(start_dim=1, end_dim=-1)
  (linear_relu_stack): Sequential(
    (0): Linear(in_features=784, out_features=512, bias=True)
    (1): ReLU()
    (2): Linear(in_features=512, out_features=512, bias=True)
    (3): ReLU()
    (4): Linear(in_features=512, out_features=10, bias=True)
  )
)
```

```
Layer: linear_relu_stack.0.weight | Size: <built-in method size of Parameter object at 0x000001601149E760> | Values : tensor([[ 0.0254, -0.0162,  0.0264, ...
[-0.0261, -0.0021, -0.0186, ...,  0.0183,  0.0327, -0.0297]],
grad_fn=<SliceBackward0>)

Layer: linear_relu_stack.0.bias | Size: <built-in method size of Parameter object at 0x0000016012E30A50> | Values : tensor([-0.0290, -0.0202], grad_fn=<SliceBackward0>)

Layer: linear_relu_stack.2.weight | Size: <built-in method size of Parameter object at 0x0000016012E30F50> | Values : tensor([[ -0.0167, -0.0054,  0.0004, ...
[ 0.0186, -0.0048,  0.0260, ..., -0.0176,  0.0183,  0.0124]],
grad_fn=<SliceBackward0>)

Layer: linear_relu_stack.2.bias | Size: <built-in method size of Parameter object at 0x0000016012E30E60> | Values : tensor([ 0.0175, -0.0165], grad_fn=<SliceBackward0>)

Layer: linear_relu_stack.4.weight | Size: <built-in method size of Parameter object at 0x0000016012E30F00> | Values : tensor([[ -0.0066, -0.0109, -0.0127, ...
[ 0.0411,  0.0313, -0.0009, ..., -0.0416,  0.0150, -0.0341]],
grad_fn=<SliceBackward0>)

Layer: linear_relu_stack.4.bias | Size: <built-in method size of Parameter object at 0x0000016012E30DC0> | Values : tensor([-0.0008,  0.0340], grad_fn=<SliceBackward0>)
```

Automatic Differentiation with Autograd

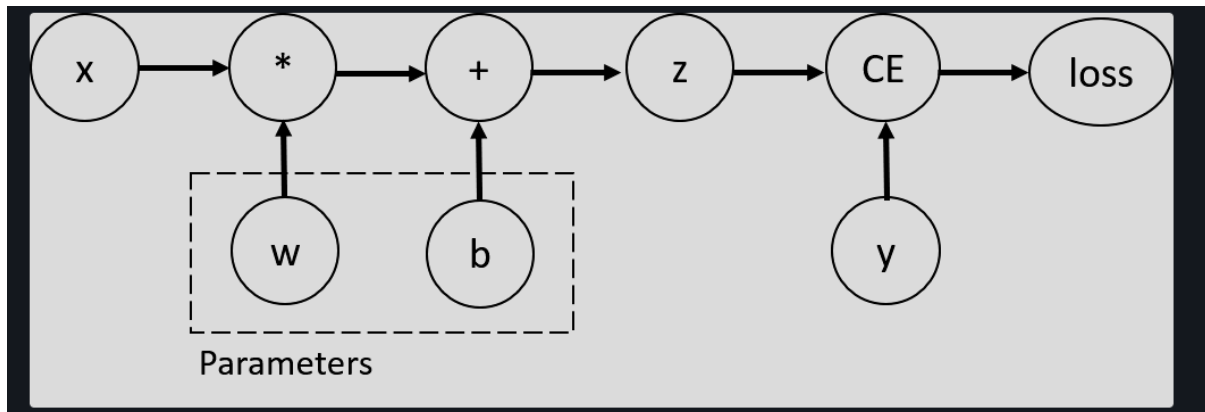
When training neural networks, the most frequently used algorithm is back propagation. In this algorithm, parameters (model weights) are adjusted according to the gradient of the loss function with respect to the given parameter.

To compute those gradients, PyTorch has a built-in differentiation engine called `torch.autograd`. It supports automatic computation of gradient for any computational graph.

Consider the simplest one-layer neural network, with input `x`, parameters `w` and `b`, and some loss function. It can be defined in PyTorch in the following manner:

```
import torch

x = torch.ones(5) # input tensor
y = torch.zeros(3) # expected output
w = torch.randn(5, 3, requires_grad=True)
b = torch.randn(3, requires_grad=True)
z = torch.matmul(x, w)+b
loss = torch.nn.functional.binary_cross_entropy_with_logits(z, y)
```



Computing Gradients:

To optimize weights of parameters in the neural network, we need to compute the derivatives of our loss function with respect to parameters, namely, we need $\frac{\partial loss}{\partial w}$ and $\frac{\partial loss}{\partial b}$ under some fixed values of x and y . To compute those derivatives, we call `loss.backward()` and then retrieve the values from `w.grad` and `b.grad`:

```
loss.backward()
print(w.grad)
print(b.grad)
```

Out:

```
tensor([[0.3110, 0.1781, 0.1174],
        [0.3110, 0.1781, 0.1174],
        [0.3110, 0.1781, 0.1174],
        [0.3110, 0.1781, 0.1174],
        [0.3110, 0.1781, 0.1174]])
tensor([0.3110, 0.1781, 0.1174])
```

Optimizing Model Parameters

Now that we have a model and data it's time to train, validate and test our model by optimizing its parameters on our data. Training a model is an iterative process; in each iteration the model makes a guess about the output, calculates the error in its guess (*loss*), collects the derivatives of the error with respect to its parameters, and **optimizes** these parameters using gradient descent.

```

import torch
from torch import nn
from torch.utils.data import DataLoader
from torchvision import datasets
from torchvision.transforms import ToTensor

training_data = datasets.FashionMNIST(
    root="data",
    train=True,
    download=True,
    transform=ToTensor()
)

test_data = datasets.FashionMNIST(
    root="data",
    train=False,
    download=True,
    transform=ToTensor()
)

train_dataloader = DataLoader(training_data, batch_size=64)
test_dataloader = DataLoader(test_data, batch_size=64)

class NeuralNetwork(nn.Module):
    def __init__(self):
        super().__init__()
        self.flatten = nn.Flatten()
        self.linear_relu_stack = nn.Sequential(
            nn.Linear(28*28, 512),
            nn.ReLU(),
            nn.Linear(512, 512),
            nn.ReLU(),
            nn.Linear(512, 10),
        )

    def forward(self, x):
        x = self.flatten(x)
        logits = self.linear_relu_stack(x)
        return logits

model = NeuralNetwork()

```

Out:

```

0%|          | 0.00/26.4M [00:00<?, ?B/s]
0%|          | 65.5k/26.4M [00:00<01:12, 365kB/s]
1%|█         | 229k/26.4M [00:00<00:38, 685kB/s]
3%|██        | 918k/26.4M [00:00<00:12, 2.12MB/s]
14%|████     | 3.67M/26.4M [00:00<00:03, 7.30MB/s]
36%|██████   | 9.63M/26.4M [00:00<00:01, 16.6MB/s]
59%|███████  | 15.5M/26.4M [00:01<00:00, 22.0MB/s]
82%|████████ | 21.6M/26.4M [00:01<00:00, 25.0MB/s]
100%|█████████| 26.4M/26.4M [00:01<00:00, 19.4MB/s]

0%|          | 0.00/29.5k [00:00<?, ?B/s]
100%|█████████| 29.5k/29.5k [00:00<00:00, 325kB/s]

0%|          | 0.00/4.42M [00:00<?, ?B/s]
1%|█         | 65.5k/4.42M [00:00<00:12, 362kB/s]
5%|██        | 229k/4.42M [00:00<00:06, 681kB/s]
20%|████     | 885k/4.42M [00:00<00:01, 2.02MB/s]
81%|███████  | 3.57M/4.42M [00:00<00:00, 7.08MB/s]
100%|█████████| 4.42M/4.42M [00:00<00:00, 6.09MB/s]

0%|          | 0.00/5.15k [00:00<?, ?B/s]
100%|█████████| 5.15k/5.15k [00:00<00:00, 32.1MB/s]

```

Hyperparameters:

Hyperparameters are adjustable parameters that let you control the model optimization process. Different hyperparameter values can impact model training and convergence rates.

We define the following hyperparameters for training:

- **Number of Epochs** - the number of times to iterate over the dataset
- **Batch Size** - the number of data samples propagated through the network before the parameters are updated
- **Learning Rate** - how much to update models parameters at each batch/epoch. Smaller values yield slow learning speed, while large values may result in unpredictable behavior during training.

```
learning_rate = 1e-3
batch_size = 64
epochs = 5
```

Optimization Loop:

Once we set our hyperparameters, we can then train and optimize our model with an optimization loop. Each iteration of the optimization loop is called an **epoch**.

Each epoch consists of two main parts:

- **The Train Loop** - iterate over the training dataset and try to converge to optimal parameters.
- **The Validation/Test Loop** - iterate over the test dataset to check if model performance is improving.

Loss Function:

When presented with some training data, our untrained network is likely not to give the correct answer. Loss function measures the degree of dissimilarity of obtained result to the target value, and it is the loss function that we want to minimize during training. To calculate the loss, we make a prediction using the inputs of our given data sample and compare it against the true data label value.

Common loss functions include [nn.MSELoss](#) (Mean Square Error) for regression tasks, and [nn.NLLLoss](#) (Negative Log Likelihood) for classification. [nn.CrossEntropyLoss](#) combines `nn.LogSoftmax` and `nn.NLLLoss`.

We pass our model's output logits to `nn.CrossEntropyLoss`, which will normalize the logits and compute the prediction error.

```
# Initialize the loss function
loss_fn = nn.CrossEntropyLoss()
```

Optimizer:

Optimization is the process of adjusting model parameters to reduce model error in each training step. Optimization algorithms define how this process is performed (in this example we use Stochastic Gradient Descent). All optimization logic is encapsulated in the optimizer object. Here, we use the SGD optimizer; additionally, there are many [different optimizers](#) available in PyTorch such as ADAM and RMSProp, that work better for different kinds of models and data.

We initialize the optimizer by registering the model's parameters that need to be trained, and passing in the learning rate hyperparameter.

```
optimizer = torch.optim.SGD(model.parameters(), lr=learning_rate)
```

Inside the training loop, optimization happens in three steps:

- Call `optimizer.zero_grad()` to reset the gradients of model parameters. Gradients by default add up; to prevent double-counting, we explicitly zero them at each iteration.

- Backpropagate the prediction loss with a call to `loss.backward()`. PyTorch deposits the gradients of the loss w.r.t. each parameter.
- Once we have our gradients, we call `optimizer.step()` to adjust the parameters by the gradients collected in the backward pass.

Final Implementation:

```
def train_loop(dataloader, model, loss_fn, optimizer):
    size = len(dataloader.dataset)
    # Set the model to training mode - important for batch normalization and dropout layers
    # Unnecessary in this situation but added for best practices
    model.train()
    for batch, (X, y) in enumerate(dataloader):
        # Compute prediction and loss
        pred = model(X)
        loss = loss_fn(pred, y)

        # Backpropagation
        loss.backward()
        optimizer.step()
        optimizer.zero_grad()

        if batch % 100 == 0:
            loss, current = loss.item(), batch * batch_size + len(X)
            print(f"loss: {loss:>7f} [{current:>5d}/{size:>5d}]")

def test_loop(dataloader, model, loss_fn):
    # Set the model to evaluation mode - important for batch normalization and dropout layers
    # Unnecessary in this situation but added for best practices
    model.eval()
    size = len(dataloader.dataset)
    num_batches = len(dataloader)
    test_loss, correct = 0, 0

    # Evaluating the model with torch.no_grad() ensures that no gradients are computed during test mode
    # also serves to reduce unnecessary gradient computations and memory usage for tensors with requires_grad=True
    with torch.no_grad():
        for X, y in dataloader:
            pred = model(X)
            test_loss += loss_fn(pred, y).item()
            correct += (pred.argmax(1) == y).type(torch.float).sum().item()

    test_loss /= num_batches
    correct /= size
    print(f"Test Error: \n Accuracy: {(100*correct)/size:.1f}%, Avg loss: {test_loss:>8f} \n")
```

We initialize the loss function and optimizer, and pass it to `train_loop` and `test_loop`.

```
loss_fn = nn.CrossEntropyLoss()
optimizer = torch.optim.SGD(model.parameters(), lr=learning_rate)

epochs = 10
for t in range(epochs):
    print(f"Epoch {t+1}\n-----")
    train_loop(train_dataloader, model, loss_fn, optimizer)
    test_loop(test_dataloader, model, loss_fn)
print("Done!")
```

Out:

```
Epoch 1
-----
loss: 2.312999 [ 64/60000]
loss: 2.304540 [ 6464/60000]
loss: 2.282702 [12864/60000]
loss: 2.271572 [19264/60000]
loss: 2.260906 [25664/60000]
loss: 2.237435 [32064/60000]
loss: 2.243402 [38464/60000]
loss: 2.212108 [44864/60000]
loss: 2.215706 [51264/60000]
loss: 2.178139 [57664/60000]
Test Error:
Accuracy: 39.7%, Avg loss: 2.176790

Epoch 2
-----
loss: 2.188591 [ 64/60000]
loss: 2.185433 [ 6464/60000]
loss: 2.128148 [12864/60000]
loss: 2.139430 [19264/60000]
loss: 2.099780 [25664/60000]
loss: 2.045364 [32064/60000]
```

Lab Task:

- Implement ANN to classify the CIFAR-10 Dataset. You must perform the following steps:
 - Import necessary libraries, download the dataset, prepare dataloader, apply required transformations (to tensor and normalisation) to your data. Also display a sample image from your dataset.
 - Implement Neural Network. Your neural network must contain 5 hidden layers; adjust parameters accordingly.
 - Train the model and run for 30, 50 and 100 epochs one by one. Record the loss and accuracy values for each. Print the results.
- Repeat the task for MNIST Dataset.